

Assignment 4 — Automated Insight Generation

1. Problem Statement

Build an **Auto-Analytics Engine** that ingests a small district-level healthcare performance CSV and automatically identifies **trends, outliers, correlations**, and produces **human-readable insights**.

The engine must be **general-purpose** — no hardcoded district-specific narratives.

Example insight

"ANC Coverage in Ahmedabad dropped by 19% compared to the previous month."

2. Dataset

2.1 Sample dataset (provided inline)

```
month,district,anc_coverage,institutional_delivery,immunization,high_risk_cases
2026-07,Ahmedabad,85,91,93,10
2026-08,Ahmedabad,69,90,92,13
2026-07,Surat,81,87,91,13
2026-08,Surat,83,89,92,12
2026-07,Vadodara,90,93,96,7
2026-08,Vadodara,91,94,97,6
2026-07,Rajkot,79,85,89,15
2026-08,Rajkot,80,86,90,14
2026-07,Mehsana,84,89,92,11
2026-08,Mehsana,42,88,91,28
2026-07,Bhavnagar,77,82,87,17
2026-08,Bhavnagar,79,84,89,15
```

Patterns to discover:

- **Ahmedabad ANC:** 85 → 69 (−18.8%) — clear trend.
- **Mehsana ANC:** 42 — extreme outlier vs state mean ~ 76.
- **Mehsana high_risk_cases:** 11 → 28 — correlated anomaly.

2.2 Required schema

Column	Example	Notes
month	2026-07-01	Monthly

district	"Ahmedabad"	
anc_coverage	85	%
institutional_delivery	91	%
immunization	93	%
high_risk_cases	10	raw count

Granularity: 1 row per (district, month) . Recommended minimum: **≥3 months** per district for trend detection, **≥10 districts** for correlation stability.

3. Tasks

Part A — Data Loading & Validation

- Load the CSV with Pandas.
- Print `head()` , `info()` , and a missing-value count per column.
- Provide UI filters (any lightweight Python UI framework of your choice) for **district**, **month**, and **indicator**.

Part B — Trend Detection

- For each (district, indicator) pair, compute $\text{pct_change} = (\text{current} - \text{previous}) / \text{previous} * 100$.
- Flag as `is_significant` if $|\text{pct_change}| \geq \text{threshold}$. Default threshold = 10% (configurable in UI).
- Output a list of flagged trends.

Part C — Outlier Detection

- For each indicator, compute the **IQR rule**: outlier if $\text{value} > Q3 + 1.5 * \text{IQR}$ OR $\text{value} < Q1 - 1.5 * \text{IQR}$.
- Alternatively: **Z-score** with threshold = 3 (configurable).
- List each outlier with `district` , `indicator` , `month` , `value` .

Part D — Correlation Detection

- Compute **Pearson correlation matrix** across the numerical indicators.
- Flag pairs with $|r| \geq 0.70$ (configurable threshold).
- Note: with only 2 months × 6 districts, true correlation is fragile — state this limitation.

Part E — Automated Insight Generation

- Generate insights dynamically. **Templated copy is OK; numeric values must come from the data.**
- Each insight row must have:
 - `insight_id` (auto-numbered)
 - `type` $\in \{ \text{trend}, \text{outlier}, \text{correlation}, \text{threshold_breach} \}$
 - `indicator`
 - `entity` (district)
 - `period` (month)
 - `metric` and `change` (numeric)
 - `severity` $\in \{ \text{Low}, \text{Medium}, \text{High} \}$
 - `explanation` (auto-generated sentence)

Part F — UI & Visualization

- Use any lightweight Python UI framework of your choice.
- Show: insight list, severity counts (bar chart), correlation heatmap, per-district line chart.
- Filters must be live.

4. Output Specification

4.1 Insights CSV

```
insight_id,type,indicator,entity,period,value,prev_value,change_pct,severity,expla
INS-0001,trend,anc_coverage,Ahmedabad,2026-08,69,85,-18.8,High,"Ahmedabad ANC cove
INS-0002,outlier,anc_coverage,Mehsana,2026-08,42,76,-44.7,High,"Mehsana ANC covera
INS-0003,correlation,anc_coverage:high_risk_cases,Mehsana,2026-07..08,r=-0.98,n/a,
```

4.2 Correlation matrix

Standard `pandas.DataFrame.corr()` output, serialized as CSV.

5. Functional Requirements

1. CSV loads; missing-value report printed.
2. Filters work in UI for district, month, indicator.
3. Trend detection with configurable threshold (default $\pm 10\%$).
4. Outlier detection via IQR or Z-score (configurable).
5. Pearson correlation matrix emitted; pairs with `|r|  $\geq 0.70$` flagged.
6. Insights emitted with all required fields.
7. Severity levels: Low / Medium / High.

8. UI displays insights + severity counts + ≥ 1 visualization.

6. Technical Constraints

- **Backend:** Python only.
- **Frontend:** any lightweight Python UI framework. No separate backend API.
- **Libs allowed:** Python data-science stack of your choice (e.g., `pandas` , `numpy` , `scipy/stats` libs).
- All thresholds **must be configurable** via UI sliders.

7. Deliverables

- Source code (`.py` and/or `.ipynb`).
- `requirements.txt` .
- Dataset CSV.
- `README.md` with run instructions.
- Screenshots or sample insights JSON.

8. Example Output

Ahmedabad — anc_coverage — 2026-08 — 69 (prev 85), Δ -18.8% — Severity: **HIGH** "ANC Coverage in Ahmedabad dropped by 18.8% compared to the previous month, exceeding the 10% significant-change threshold."

Mehsana — anc_coverage — 2026-08 — 42 — Severity: **HIGH** "Mehsana's ANC coverage of 42% is 3.1σ below the state mean (76), flagging for review."

9. Evaluation Rubric (10 marks)

#	Criterion	Marks
1	Data loading + validation + filters in UI	1.5
2	Trend detection (configurable threshold)	2.0
3	Outlier detection (IQR or Z-score, configurable)	2.0
4	Correlation detection with threshold flagging	1.5
5	Insights: dynamic, structured fields, severity Low/Medium/High	2.0
6	UI + visualizations (line, bar, heatmap)	1.0
Total		10

9.1 Pass criteria

- $\geq 6/10$ overall AND
- ≥ 1 mark on Criterion 5 (insight generation is the core deliverable).

9.2 Scoring scale

0 = Not done · 0.5 = Started · 1.0 = Partial · 1.5 = Mostly done · 2.0 = Complete.

10. Notes for Candidates

- **No hardcoded narratives.** "ANC coverage in Ahmedabad dropped" with hardcoded values is an automatic fail. Templates are fine; numbers must be data-driven.
- **2 months × 6 districts = 12 rows.** Most Pearson correlations will be unstable. State this in the README; show the matrix honestly.
- **Outlier vs trend are different insight types.** Don't conflate them.
- **Severity must be derived from data**, not a magic number like `if change > 50: HIGH`. Use the configurable threshold